

GUÍA TÉCNICA Y RÚBRICA DE EVALUACIÓN FINAL

Proyecto de Sistemas Distribuidos – Ciencia de la Computación

Este documento establece los requisitos mínimos, la estructura obligatoria y los criterios de evaluación para el informe técnico final del proyecto de Sistemas Distribuidos. El informe deberá adjuntarse directamente en el repositorio Git del proyecto y constituirá la base documental para la defensa técnica y calificación.

1. ENTREGABLES OBLIGATORIOS

1. **Informe técnico** en formato Markdown (`.md`) o PDF, ubicado en la raíz del repositorio o en la carpeta `/docs`.
2. **Instrucciones de ejecución directa** documentadas paso a paso y verificables en un entorno limpio.
3. **Video de demostración** (máximo 15 minutos) alojado obligatoriamente en **minube.uh.cu**, con enlace público o accesible para la cátedra, referenciado en el informe.
4. **Código fuente y configuración** del sistema distribuido, completamente funcional y versionado en Git.

Condiciones de aprobación: La ausencia de cualquiera de estos entregables, o una calidad insuficiente en su contenido, ejecución o demostración, implicará **suspense automático** del proyecto. No se aceptarán entregas parciales ni ampliaciones de plazo para subsanar requisitos críticos.

2. ESTRUCTURA Y CONTENIDOS DEL INFORME TÉCNICO

El informe debe abordar explícitamente los siguientes dominios de diseño distribuido. Cada sección deberá incluir justificación técnica, decisiones de arquitectura, trade-offs evaluados y referencias al código correspondiente.

2.1 Arquitectura y Organización del Sistema

- Planteamiento del problema de diseño y justificación de la arquitectura elegida.
- Organización lógica y física del sistema distribuido.
- Definición de roles (ej. coordinador, worker, proxy, gateway, stateful node, stateless node, etc.).
- Distribución de servicios en ambas redes de Docker: topología, aislamiento, puertos expuestos, volúmenes y política de red.

2.2 Procesos y Modelo de Concurrency

- Tipos de procesos/servicios que componen el sistema.
- Agrupación de procesos por instancia o contenedor según la arquitectura.
- Patrón de diseño de concurrency y ejecución: asíncrono, multihilo, multiproceso, event-loop, actores, o subconjunto híbrido. Justificación de la elección frente al desempeño esperado.

2.3 Comunicación y Protocolos

- Mecanismo de comunicación elegido (RPC, REST, gRPC, sockets TCP/UDP, colas de mensajes, objetos remotos, pub/sub, etc.).

- Flujo cliente-servidor y servidor-servidor: serialización, codificación, manejo de latencia y timeouts.
- Comunicación entre procesos dentro de la misma instancia (IPC, shared memory, pipes, sockets locales) si aplica.

2.4 Coordinación y Sincronización

- Mecanismos de sincronización de acciones distribuidas (relojes lógicos, vectores, sincronización de estados).
- Acceso exclusivo a recursos compartidos: algoritmos de exclusión mutua distribuida, manejo de condiciones de carrera.
- Toma de decisiones distribuidas: consenso, quórum, elección de líder, votación o protocolos específicos (Raft, Paxos, Bully, etc.).

2.5 Nombrado y Localización

- Esquema de identificación de datos y servicios (UUID, DNS, hashes, rutas lógicas).
- Ubicación física/lógica de los recursos y estrategia de despliegue.
- Mecanismos de descubrimiento y enrutamiento (Service Discovery, DNS interno, registros distribuidos, proxies inversos, tablas de enrutamiento).

2.6 Consistencia y Replicación

- Estrategia de distribución de datos (sharding, particionamiento, replicación completa o parcial).
- Número de réplicas por dato/servicio y política de escritura/lectura.
- Modelo de consistencia adoptado (fuerte, eventual, causal, session, etc.) y garantías ofrecidas tras una actualización.
- **REQUISITO CRÍTICO:** Documentación detallada de **cómo se maneja la replicación y cómo se recupera el sistema ante una partición de red** para **todos los nodos que gestionan estado** (bases de datos, logs, cachés persistentes, colas con ack, nodos de configuración, etc.). Debe incluir:
 - Detección de partición (heartbeats, timeouts, fencing tokens).
 - Resolución de split-brain y prevención de escrituras conflictivas.
 - Protocolo de recuperación post-partición (reconciliación, replay de logs, sincronización incremental, resolución de conflictos).
 - Evidencia de coherencia de estado tras la reintegración de nodos.

2.7 Tolerancia a Fallos

- Estrategia de respuesta a errores (retry con backoff, circuit breakers, dead letter queues, rollback, compensación).
- Nivel de tolerancia esperado ($n+1$, quórum, tolerancia a fallos bizantinos o no, grado de disponibilidad objetivo).
- Manejo de fallos parciales: nodos caídos temporalmente, nodos nuevos que se incorporan (join protocol), reequilibrio de carga y reasignación de responsabilidades.

2.8 Seguridad

- Seguridad en la comunicación: cifrado en tránsito (TLS/mTLS), validación de certificados, prevención de replay/man-in-the-middle.

- Seguridad en el diseño: principios de mínimo privilegio, aislamiento de fronteras de confianza, hardening de contenedores.
 - Autenticación y autorización: mecanismos implementados (JWT, OAuth2, tokens, RBAC/ABAC, API keys), gestión de credenciales y rotación.
-

3. INSTRUCCIONES DE EJECUCIÓN DIRECTA

El informe debe incluir una sección titulada **Ejecución del Proyecto** con:

- Requisitos previos (Docker, Docker Compose, versiones de Python/Java/Go, variables de entorno, recursos mínimos).
 - Comandos exactos para clonar, construir, levantar y detener el sistema.
 - Verificación de salud (**healthcheck** endpoints, logs esperados, comandos de validación).
 - Ejemplo de flujo de trabajo completo desde cero hasta el estado operativo.
 - La ejecución debe ser **reproducible sin intervención manual** (scripts, Makefile, docker-compose bien definido).
-

4. REQUISITOS DEL VIDEO DE DEMOSTRACIÓN

- **Duración máxima:** 15 minutos.
- **Plataforma de alojamiento obligatorio:** **minube.uh.cu**. El video debe estar subido a esta plataforma institucional con permisos de visualización habilitados para la cátedra.
- **Contenido obligatorio:**
 1. Levantamiento completo del sistema desde cero.
 2. Navegación por la funcionalidad principal de la aplicación.
 3. Ejecución del protocolo de pruebas de tolerancia descrito en la sección 5.
 4. Verificación en vivo de que la replicación y consistencia se mantienen.
- El video debe mostrar terminal/logs, interfaz de usuario (si aplica) y métricas de estado. Subtítulos o narración clara son recomendados.
- El enlace a **minube.uh.cu** debe estar referenciado explícitamente en el informe técnico y en un archivo **video_link.txt** en la raíz del repositorio.

Videos alojados en otras plataformas (YouTube, Drive, Vimeo, etc.) **no serán evaluados**, salvo autorización expresa y previa de la cátedra.

5. PRUEBAS OBLIGATORIAS DE TOLERANCIA Y CONSISTENCIA

Durante el video y documentado en el informe, se debe ejecutar y validar el siguiente escenario **para cada tipo de nodo que gestiona estado** (ej. DB primaria/replica, log service, state manager, etc.):

1. Levantar **al menos 3 nodos** del mismo tipo que gestionen estado.
2. Apagar **2 de ellos** de forma abrupta o controlada.
3. Levantar **1 nodo nuevo** del mismo tipo.
4. Apagar **el nodo restante** de los iniciales.
5. **Verificar explícitamente** que:
 - Los datos escritos antes/durante la prueba se replican correctamente.

- No hay pérdida de estado ni divergencia irreversible.
- El sistema mantiene consistencia según el modelo declarado.
- Los nodos nuevos se sincronizan correctamente con el estado actual.
- Los logs/métricas reflejan la detección de caída, reelección/quórum y reconciliación.

La validación debe incluir comandos de lectura/escritura, checksums, comparación de estados o queries explícitas que demuestren coherencia. Simulaciones teóricas o afirmaciones sin evidencia ejecutable no serán aceptadas.

Nota de política académica: La falta de **cualquiera** de los entregables, la omisión del escenario de prueba, la ausencia de documentación sobre replicación/partición, o una calidad técnica que no demuestre comprensión profunda de sistemas distribuidos, resultará en **suspense directo**. No se otorgará puntuación parcial por componentes críticos incompletos.

Checklist de Validación Pre-Entrega

- ☐ Informe completo con las 8 secciones técnicas.
- ☐ Sección explícita de replicación y recuperación ante partición para nodos stateful.
- ☐ Instrucciones de ejecución paso a paso y verificables.
- ☐ Video <=15 min subido a **minube.uh.cu** con levantamiento, funcionalidad y prueba de fallos.
- ☐ Enlace al video en minube.uh.cu accesible y referenciado en el informe y en **video_link.txt**.
- ☐ Prueba 3 up -> 2 down -> 1 up -> 1 down ejecutada y verificada para **cada** nodo de estado.
- ☐ Código en Git, bien versionado y funcional.

La omisión de cualquiera de estos puntos implica **suspense automático**. Se recomienda realizar una revisión cruzada entre los miembros del equipo antes de la entrega final.